

Day 20 : Number, Math and String

Number

1. Introduction: The Unified Number Type

In JavaScript, there is only one type for numbers: `number`. This single type is used to represent both integers (whole numbers like `10`, `-50`) and floating-point numbers (decimals like `3.14`, `-0.5`).

The Core Standard: All numbers in JavaScript are implemented as **64-bit double-precision floating-point numbers**, following the international **IEEE 754 standard**. This is the same format used for `double` in languages like C++ and Java.

```
let integer = 100;
let float = 99.5;

console.log(typeof integer); // "number"
console.log(typeof float);  // "number"
```

2. Creating Numbers

You can create numbers in several ways:

- **Standard Literals:**

```
let a = 25;    // Integer
let b = 12.34; // Floating-point
```

- **Exponential Notation (`e`):** A shorthand for writing very large or very small numbers.

```
let billion = 1e9; // 1 followed by 9 zeros → 1000000000
let tiny = 5e-6; // 5 / 10^6 → 0.000005
```

- **Other Bases (Hex, Binary, Octal):** You can also represent numbers in other numeral systems.

```
let hex = 0xFF; // Hexadecimal (base 16) → 255 in decimal
let binary = 0b1010; // Binary (base 2) → 10 in decimal
let octal = 0o77; // Octal (base 8) → 63 in decimal
```

3. The "Gotcha": Floating-Point Inaccuracy

Because numbers are stored in a binary floating-point format (base-2), they cannot perfectly represent all decimal fractions (base-10). This leads to the most famous "gotcha" in JavaScript math.

```
console.log(0.1 + 0.2); // Outputs: 0.30000000000000004
console.log(0.1 + 0.2 === 0.3); // false
```

Why? The numbers `0.1` and `0.2` (and `0.3`) have infinitely repeating representations in binary, similar to how $\frac{1}{3}$ is `0.333...` in decimal. The computer has to round them off to fit them into the 64-bit storage. These tiny rounding errors accumulate.

How to handle this:

1. **For financial calculations:** Never use floating-point numbers. Work with integers (e.g., store money in cents).
2. **For display:** Use the `.toFixed()` method to round the result to a specific number of decimal places.
3. **For comparison:** Check if two numbers are "close enough" using `Number.EPSILON`.

4. Special Numeric Values

There are three special values that are technically of type `number`.

- **Infinity** : Represents a value larger than the largest possible number. It results from division by zero or exceeding `Number.MAX_VALUE`.

```
console.log(1 / 0);      // Infinity
console.log(-1 / 0);    // -Infinity
console.log(typeof Infinity); // "number"
```

- **Infinity** : Represents a value smaller than the smallest possible number.
- **NaN (Not a Number)**: Represents the result of an invalid or undefined mathematical operation. It's the error code for failed math.**Key Property of NaN** : It is the only value in JavaScript that is not equal to itself.

```
console.log("hello" / 2); // NaN
console.log(Math.sqrt(-1)); // NaN
console.log(typeof NaN); // "number"
```

```
console.log(NaN === NaN); // false
```

5. Important **Number** Properties and Methods

The `Number` object provides several useful constants and methods.

A. **Number** Constants (Limits of the Type)

- **Number.MAX_VALUE** : The largest positive number that can be represented (~1.8e+308).
- **Number.MIN_VALUE** : The smallest positive number closest to zero.
- **Number.MAX_SAFE_INTEGER** : The largest integer that can be safely represented without losing precision ($2^{53} - 1$). **This is the one you should care about for integer math.**
- **Number.MIN_SAFE_INTEGER** : The smallest safe integer.
- **Number.EPSILON** : The smallest interval between two representable numbers. Used for floating-point comparisons.

B. Checking Number Types

- `isNaN(value)` **(Global function)**: The classic way to check if a value is `NaN`.
Gotcha: It coerces the value first, so `isNaN("hello")` is `true`, which can be misleading.
- `Number.isNaN(value)` **(Modern - ES6)**: The better, more reliable way. It returns `true` **only if** the value is actually `NaN`. It does not perform type coercion.

```
isNaN("blue");    // true (coerces "blue" to NaN)
Number.isNaN("blue"); // false (it's a string, not NaN)

let result = 0 / 0; // result is NaN
isNaN(result);     // true
Number.isNaN(result); // true
```

- `isFinite(value)` / `Number.isFinite(value)`: Checks if a value is a real, finite number (not `Infinity`, `-Infinity`, or `NaN`). The `Number.isFinite()` version is stricter and does not coerce.
- `Number.isInteger(value)`: Checks if a value is an integer.

C. Formatting and Converting Numbers

These methods are called on a number variable.

- `.toString(base)`: Converts a number to a string. You can optionally provide a base (from 2 to 36).

```
let num = 255;
console.log(num.toString()); // "255" (base 10 - default)
console.log(num.toString(16)); // "ff" (base 16 - hexadecimal)
console.log(num.toString(2)); // "1111111" (base 2 - binary)
```

- `.toFixed(digits)`: Formats a number to a fixed number of decimal places and returns a **string**. Useful for formatting currency.

```
let price = 19.991234;
console.log(price.toFixed(2)); // "19.99"
```

- `.toFixed(digits)` : Formats a number to a specified total number of significant digits and returns a **string**.

```
let n = 123.456;  
console.log(n.toFixed(4)); // "123.5" (4 significant digits)
```

6. The **Math** Object

JavaScript provides a built-in `Math` object that has properties and methods for mathematical constants and functions. It is not a constructor; you use it directly.

- **Constants:** `Math.PI` , `Math.E`
- **Rounding:**
 - `Math.round(x)` : Standard rounding to the nearest integer.
 - `Math.floor(x)` : Rounds **down** to the nearest integer.
 - `Math.ceil(x)` : Rounds **up** to the nearest integer.
 - `Math.trunc(x)` : Removes the decimal part, leaving only the integer (truncates toward zero).
- **Other Common Functions:**
 - `Math.abs(x)` : Absolute value.
 - `Math.pow(x, y)` : `x` to the power of `y` (same as `x ** y`).
 - `Math.sqrt(x)` : Square root.
 - `Math.max(a, b, c...)` : Returns the largest of the given numbers.
 - `Math.min(a, b, c...)` : Returns the smallest.
 - `Math.random()` : Returns a pseudo-random number between 0 (inclusive) and 1 (exclusive).

```
console.log(Math.round(4.7)); // 5  
console.log(Math.floor(4.7)); // 4  
console.log(Math.ceil(4.2)); // 5
```

```
console.log(Math.max(10, -5, 100, 0)); // 100

// To get a random integer between 1 and 10 (inclusive):
let randomInt = Math.floor(Math.random() * 10) + 1;
```

The Core Tool: `Math.random()`

JavaScript's only built-in tool for randomness is `Math.random()`. You must understand its behavior perfectly to build anything else.

`Math.random()` returns a random floating-point number between `0` (inclusive) and `1` (exclusive).

This means it can return `0`, `0.1234`, `0.5`, `0.99999`, but it will **never** return `1.0`.

- **Range:** `[0, 1)`

The Goal: A Random Integer Between `min` and `max` (Inclusive)

Let's say our goal is to get a random whole number from 1 to 10. We want `1`, `2`, `3`, ..., `9`, `10`.

The formula to achieve this is:

```
Math.floor(Math.random() * (max - min + 1)) + min
```

This looks complicated, so let's build it up step-by-step to understand the intuition.

Building the Formula from First Principles

Let's stick with our goal: a random integer between `1` (**min**) and `10` (**max**).

Step 1: Scale up the range.

`Math.random()` gives us a number between `0` and `1` (e.g., `0.5`). We want a number in a much larger range. How many different numbers do we want to be able to generate?

1, 2, 3, 4, 5, 6, 7, 8, 9, 10

There are **10** possible outcomes.

The number of outcomes is always $\text{max} - \text{min} + 1$.

$10 - 1 + 1 = 10$.

So, the first step is to scale our $[0, 1)$ range up to a range of the correct size. We do this by multiplying.

```
Math.random() * 10;
```

- The lowest possible value is $0 * 10 = 0$.
- The highest possible value is $0.999... * 10 = 9.999...$.
- **Our new range is $[0, 10)$.**

This gives us a floating-point number from 0 up to (but not including) 10 .

Step 2: Get rid of the decimals.

We want whole numbers (integers). The perfect tool for this is `Math.floor()`, which always rounds **down** to the nearest whole number.

Let's see what happens when we apply `Math.floor()` to our range $[0, 10)$:

```
Math.floor(Math.random() * 10);
```

- If `Math.random()` was 0.0 , `Math.floor(0)` is 0 .
- If `Math.random()` was 0.123 , `Math.floor(1.23)` is 1 .
- If `Math.random()` was 0.999 , `Math.floor(9.99)` is 9 .

Applying `Math.floor()` has transformed our float range $[0, 10)$ into an integer range of $0, 1, 2, 3, 4, 5, 6, 7, 8, 9$.

This is close! We have 10 possible numbers, but the range is wrong. We want 1 to 10 , not 0 to 9 .

Step 3: Shift the range to the correct starting point.

Our current range is $[0, 9]$. We want $[1, 10]$.

How do you get from 0 to 1? You add 1.

How do you get from 9 to 10? You add 1.

The final step is to shift our entire range up by adding our desired minimum value (`min`).

```
Math.floor(Math.random() * 10) + 1;
```

Let's trace the outcomes:

- If the `Math.floor()` part gave us `0`, the result is `0 + 1 = 1`.
- If the `Math.floor()` part gave us `9`, the result is `9 + 1 = 10`.

This formula now correctly produces a random integer from **1 to 10, inclusive**.

The General-Purpose Function

By replacing the specific numbers `10` and `1` with `max` and `min`, we get our general-purpose formula.

```
/**
 * Generates a random integer between a minimum and maximum value (inclusive).
 * @param {number} min The minimum possible value.
 * @param {number} max The maximum possible value.
 * @returns {number} A random integer within the range.
 */
function getRandomInt(min, max) {
  // 1. Calculate the number of possible outcomes (the size of our range).
  const range = max - min + 1;

  // 2. Scale up Math.random() to create a float in the range [0, range).
  const scaled = Math.random() * range;

  // 3. Round down to get an integer in the range [0, range-1].
  const floored = Math.floor(scaled);

  // 4. Shift the range up to [min, max] by adding the minimum value.
```



```
const result = floored + min;

return result;
}

// Example usage:
console.log("Random number between 1 and 10:", getRandomInt(1, 10));
console.log("Random number between 50 and 100:", getRandomInt(50, 100));
console.log("Random dice roll (1 to 6):", getRandomInt(1, 6));
```

This function encapsulates the logic perfectly and can be reused anywhere in your project.

1. Introduction: What is a String?

A string is a primitive data type in JavaScript used to represent a sequence of characters. Anything you can type—letters, numbers, symbols, punctuation—can be part of a string. It is the primary way we work with textual data.

Key Characteristics:

- **It's a primitive:** This means strings are **immutable**.
- **It's indexed:** Each character in a string has a numerical position (index), starting from zero.
- **It's an object-like primitive:** Although it's a primitive, it has methods and properties we can use, like `.length`. JavaScript temporarily wraps it in a String object when you try to access these.

2. Creating Strings

There are three ways to create a string literal in JavaScript.

1. Single Quotes (`'...'`):

```
let singleQuoted = 'Hello, world!';
```

2. **Double Quotes ("...")**: Functionally identical to single quotes. The main reason to choose one over the other is for convenience when a string itself contains quotes.

```
let doubleQuoted = "He said, 'Hello!';" // Easy to include single quotes
let singleQuotedWithDouble = 'She replied, "Hi!";' // Easy to include double quotes
```

3. **Template Literals (`...` - ES6)**: The most powerful and modern way. They use backticks.

We will cover the special features of template literals later.

```
let templateLiteral = `This is a template literal.`;
```

3. Core Properties and Concepts

A. The `.length` Property

Every string has a `.length` property that tells you how many characters it contains.

```
let greeting = "Hello";
console.log(greeting.length); // Outputs: 5

let emptyString = "";
console.log(emptyString.length); // Outputs: 0
```

B. Accessing Individual Characters (Zero-Based Indexing)

You can access a character at a specific position using square bracket notation

`[]`. The first character is at index `0`.

```
let message = "JavaScript";
// J a v a S c r i p t
// 0 1 2 3 4 5 6 7 8 9

console.log(message[0]); // "J"
console.log(message[4]); // "S"
```

```
// To get the last character, a common pattern is used:  
console.log(message[message.length - 1]); // "t"
```

C. The Golden Rule: Strings are Immutable

This is the most critical concept. **You cannot change a string in place.** Any method that appears to modify a string will always **return a brand new string**, leaving the original untouched.

```
let name = "alex";  
  
// Let's try to change the first character.  
name[0] = "A"; // This will FAIL silently. It does nothing.  
console.log(name); // Outputs: "alex" (The original string is unchanged)  
  
// Let's use a method that "changes" the string.  
let upperName = name.toUpperCase();  
console.log(upperName); // Outputs: "ALEX" (This is a NEW string)  
console.log(name); // Outputs: "alex" (The original is still unchanged)
```

4. Common and Essential String Methods

These methods are your primary tools for working with strings. Remember, they all return **new strings**.

A. Changing Case

- `.toUpperCase()` : Returns a new string with all characters in uppercase.
- `.toLowerCase()` : Returns a new string with all characters in lowercase.

```
let whisper = "please be quiet";  
let shout = whisper.toUpperCase(); // "PLEASE BE QUIET"
```

B. Finding Substrings

- `.indexOf(substring)` : Returns the index of the **first occurrence** of a substring. If the substring is not found, it returns **1**.

- `.lastIndexOf(substring)` : Returns the index of the **last occurrence** of a substring. Returns **1** if not found.
- `.includes(substring)` **(ES6)**: Returns a boolean (`true` or `false`) indicating if the string contains the substring. This is often more readable than `indexOf` .

```
let sentence = "The quick brown fox jumps over the lazy fox.";

console.log(sentence.indexOf("fox")); // 16 (the first one)
console.log(sentence.lastIndexOf("fox")); // 40 (the last one)
console.log(sentence.indexOf("cat")); // -1 (not found)

console.log(sentence.includes("jumps")); // true
console.log(sentence.includes("cat")); // false
```

C. Extracting Substrings

- `.slice(startIndex, endIndex)` : Extracts a section of a string and returns it as a new string.
 - `startIndex` : The index where the extraction begins (inclusive).
 - `endIndex` : The index where the extraction ends (exclusive - it does not include this character).
 - You can use negative indices, which count from the end of the string.

```
let text = "JavaScript";

console.log(text.slice(0, 4)); // "Java" (from index 0 up to, but not including, 4)
console.log(text.slice(4)); // "Script" (from index 4 to the end)
console.log(text.slice(-6)); // "Script" (the last 6 characters)
```

- `.substring(startIndex, endIndex)` : Similar to `.slice()` , but it doesn't accept negative indices.
- `.substr(startIndex, length)` : **(Deprecated - Avoid)**. It works with a start index and a length, which is different from the others. Use `.slice()` instead.

D. Replacing Substrings

- `.replace(searchValue, newValue)` : Finds a `searchValue` and replaces it with `newValue`.
 - **Gotcha:** By default, it only replaces the **first occurrence**.
- `.replaceAll(searchValue, newValue)` **(ES2021)**: Replaces **all occurrences**. This is the modern, easy way to do a global replace.

```
let greeting = "hello world, hello there";

// Replaces only the first "hello"
let newGreeting = greeting.replace("hello", "hi");
console.log(newGreeting); // "hi world, hello there"

// Replaces all "hello"s
let allNewGreeting = greeting.replaceAll("hello", "hi");
console.log(allNewGreeting); // "hi world, hi there"
```

- **Old way to replace all:** Use a regular expression with the global flag (`/g`).

```
greeting.replace(/hello/g, "hi");
```

E. Cleaning Up Whitespace

- `.trim()` : Removes whitespace from both the beginning and end of a string.
- `.trimStart()` / `.trimEnd()` : Removes whitespace from only the start or end.

```
let userInput = " my-username@example.com ";
let cleanedInput = userInput.trim(); // "my-username@example.com"
```

F. Splitting a String into an Array

- `.split(separator)` : Splits a string into an array of substrings, using the `separator` to decide where to split. This is incredibly useful.

```
let csvData = "item1,item2,item3";
let items = csvData.split(","); // ["item1", "item2", "item3"]

let words = "The quick brown fox";
```

```
let word_array = words.split(" "); // ["The", "quick", "brown", "fox"]

let letters = "abc";
let letter_array = letters.split(""); // ["a", "b", "c"]
```

5. Template Literals (ES6) - The Modern Way

Template literals, created with backticks (```), are a massive improvement for working with strings.

A. Variable Interpolation (`${...}`):

This allows you to embed expressions and variables directly into a string. It's far more readable than using the `+` operator for concatenation.

- **The Old Way (Concatenation):**

```
let name = "Alice";
let age = 30;
let message = "Hello, my name is " + name + " and I am " + age + " years old.";
```

- **The New Way (Template Literals):**

```
let name = "Alice";
let age = 30;
let message = `Hello, my name is ${name} and I am ${age} years old.`;
// You can even put expressions inside:
let futureMessage = `Next year, I will be ${age + 1}.`;
```

B. Multi-line Strings:

Template literals respect newlines inside the string.

- **The Old Way:**

```
let htmlOld = '<div>\n' +
  ' <p>Hello</p>\n' +
```

```
'</div>';
```

- **The New Way:**

```
let htmlNew = `  
  <div>  
    <p>Hello</p>  
  </div>  
`;
```

